16.3.2 Alternative 2: Modeling exceptions as data

You can model the exception as a class instead of handling it directly in the stream. This is the recommended approach for streams and programming in general. Create a class that wraps the `ArithmeticException` and a method that handles it:

```
class ExceptionsSpec ... {
  ...

  case class Result(value: Int, isValid: Boolean, message: String = "")  ❶

  def properHandler(elem: Int): Result =                                 ❷
    try {
      Result(100 / elem, true)
    } catch {
      case ex: ArithmeticException =>
        Result(elem, false, s"failed processing [$elem] with $ex.")
    }
  ...
}
```

❶ Business class to wrap exceptions

❷ Method modeling ArithmeticException as data

If you convert the result you had before (an `Int` or an `ArithmeticException`) into a `Result`, you are in a much better situation. If a method can throw exceptions, that is a problem that the risky method should solve, not the stream or another client.

When you work with streams, they are often sent to one sink or another depending on the content of an element. For example, if the element contains an exception, you may want to divert it to another sink.